

Responding to events with state machines

Levels, edges, qualified events, and remembered context



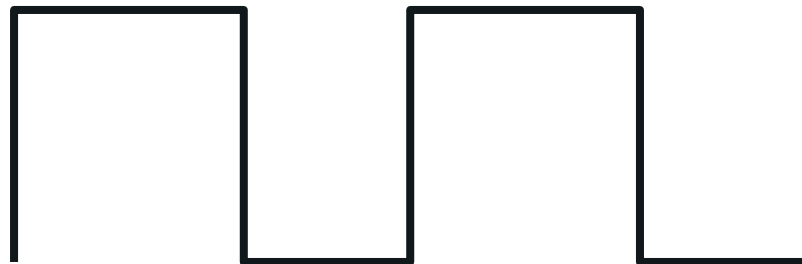
From Vehicle Presence to a Digital Signal



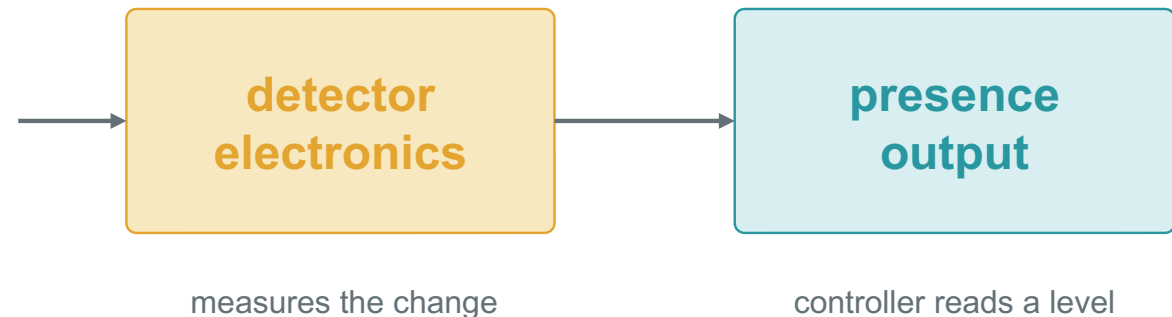
wire loop cut into pavement

**A vehicle changes the loop's inductance.
Resulting in a high signal when a vehicle is present**

vehicle present



no vehicle present



Signal Levels

vehicle present

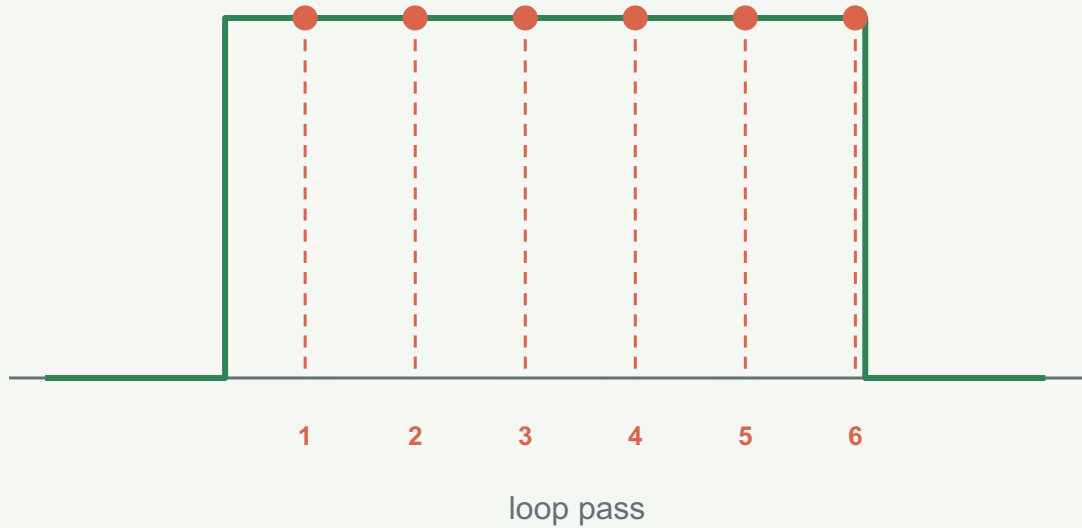
vehicle absent



present = HIGH for how long?

Polling a Level

detector input



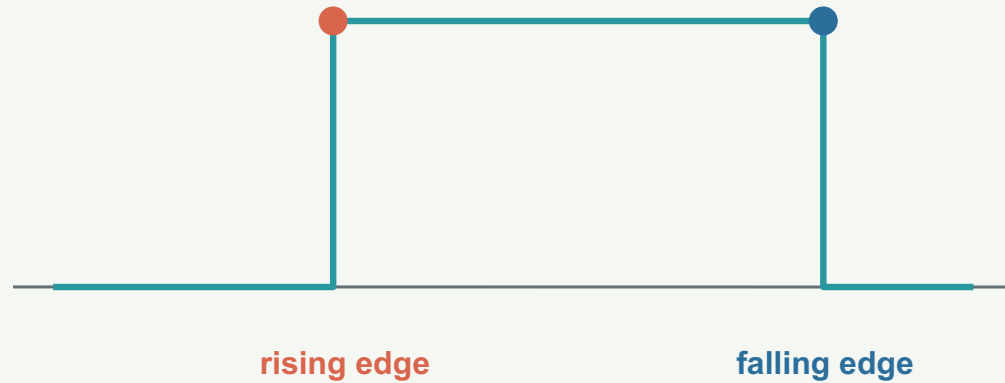
```
bool present = digitalRead(DETECTOR_PIN);  
  
if (present) {  
    executeFunction();  
}
```

Each sample is HIGH, so the branch executes on every pass.



Edge Detection

signal



previous	current	edge
LOW	LOW	none
LOW	HIGH	rising
HIGH	HIGH	none
HIGH	LOW	falling

Complete the two conditions

```
bool current = digitalRead(DETECTOR_PIN);
```

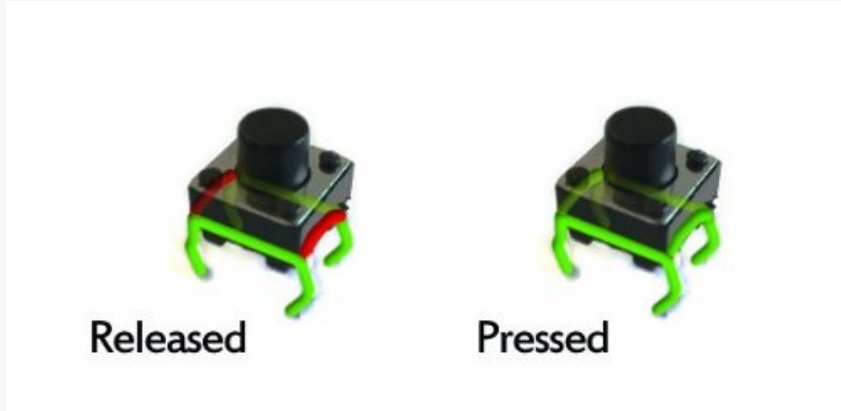
```
bool rising = _____;
```

```
bool falling = _____;
```

```
previous = current;
```

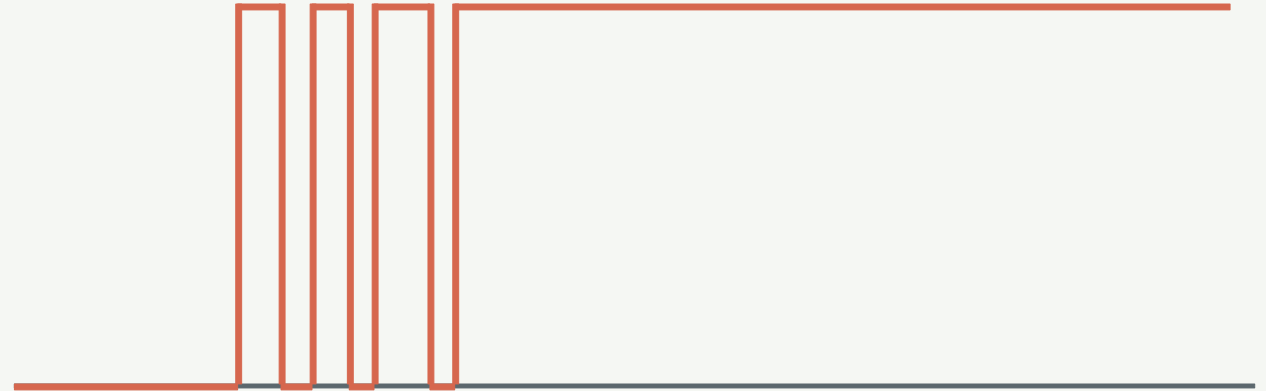
```
rising = !previous && current;  
falling = previous && !current;
```

Contact Bounce



one press

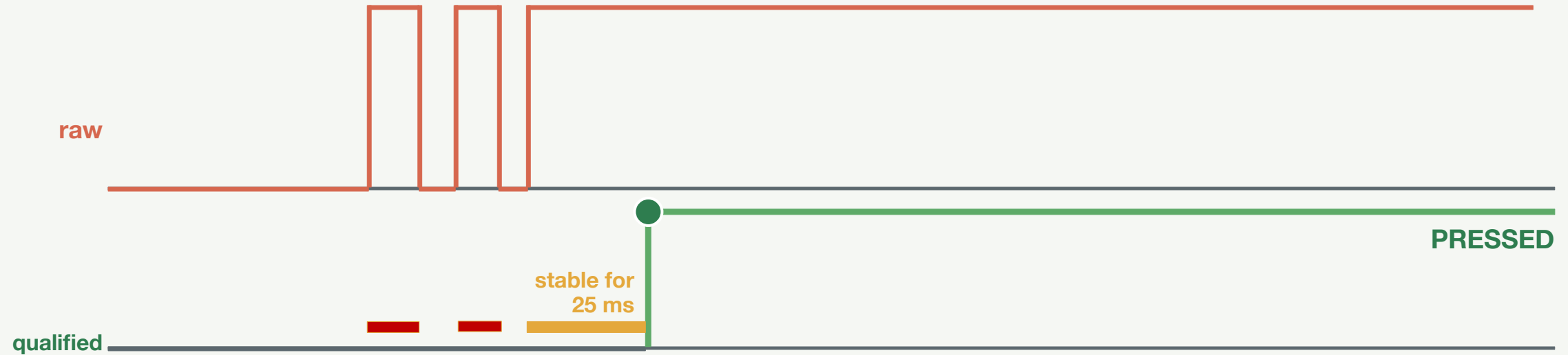
raw digital reading



several apparent edges

Contact bounce can produce multiple electrical edges from one mechanical press.

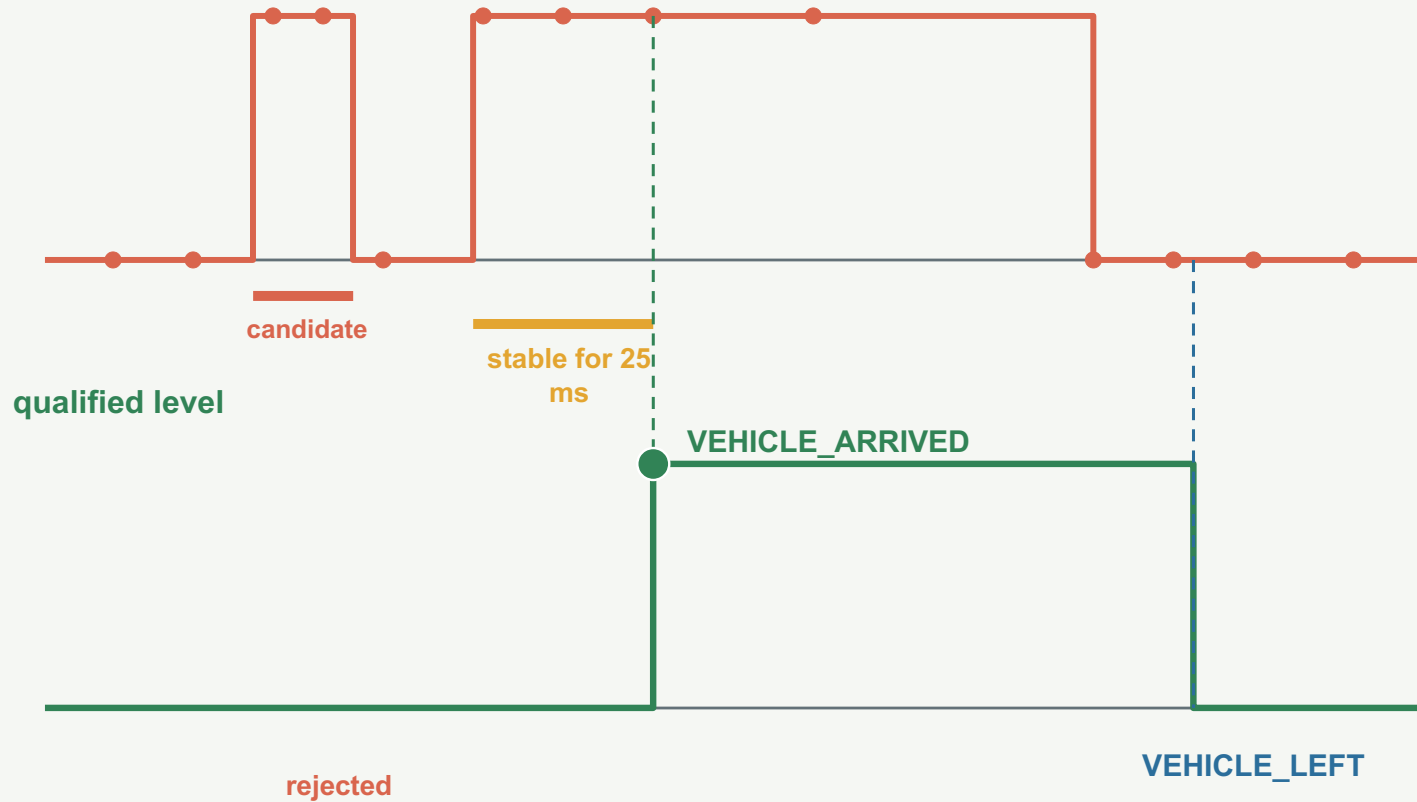
Event Qualification



Reject events that are below a time interval threshold

Event Extraction

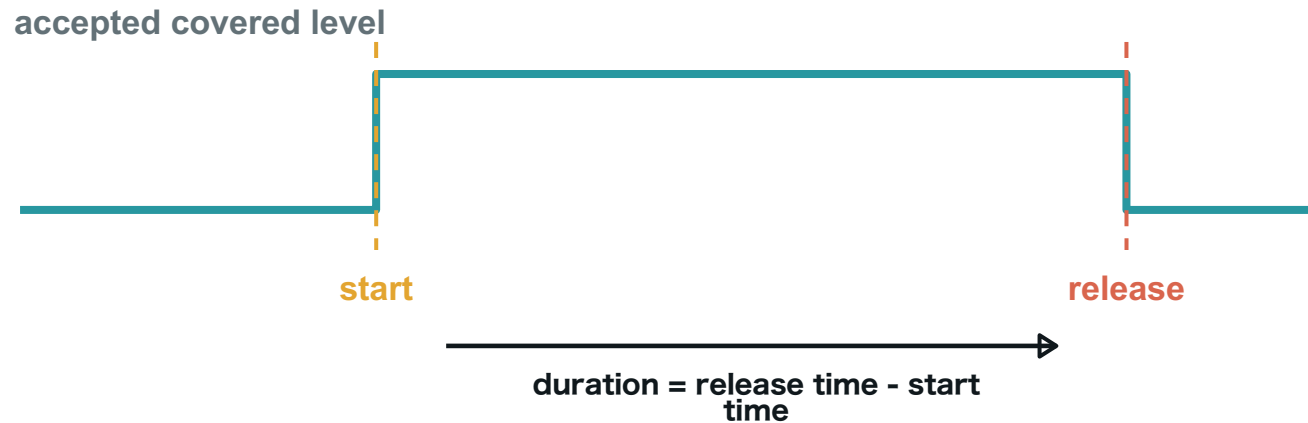
raw input



```
if (raw != candidate) {  
    candidate = raw;  
    candidateSince = now;  
}  
  
if (candidate != qualified &&  
    now - candidateSince >= 25) {  
    qualified = candidate;  
    if (qualified) emit(VEHICLE_ARRIVED);  
    else emit(VEHICLE_LEFT);  
}
```

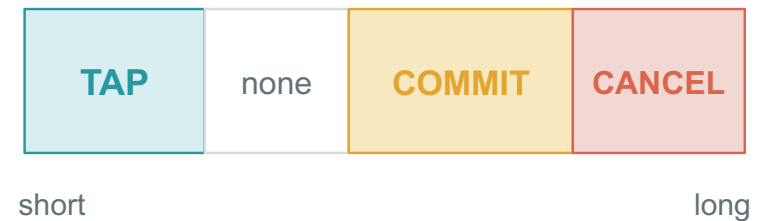
raw ➤ candidate ➤ qualified ➤ event

Duration-Based Events



```
if (covered && !wasCovered) {
  wasCovered = true;
  startedMs = nowMs;
  cancelEmitted = false;
} else if (covered && !cancelEmitted &&
  nowMs - startedMs >= cancelMs) {
  cancelEmitted = true;
  return CANCEL;
} else if (!covered && wasCovered) {
  wasCovered = false;
  if (cancelEmitted) return NONE;
  return classify(nowMs - startedMs);
}
```

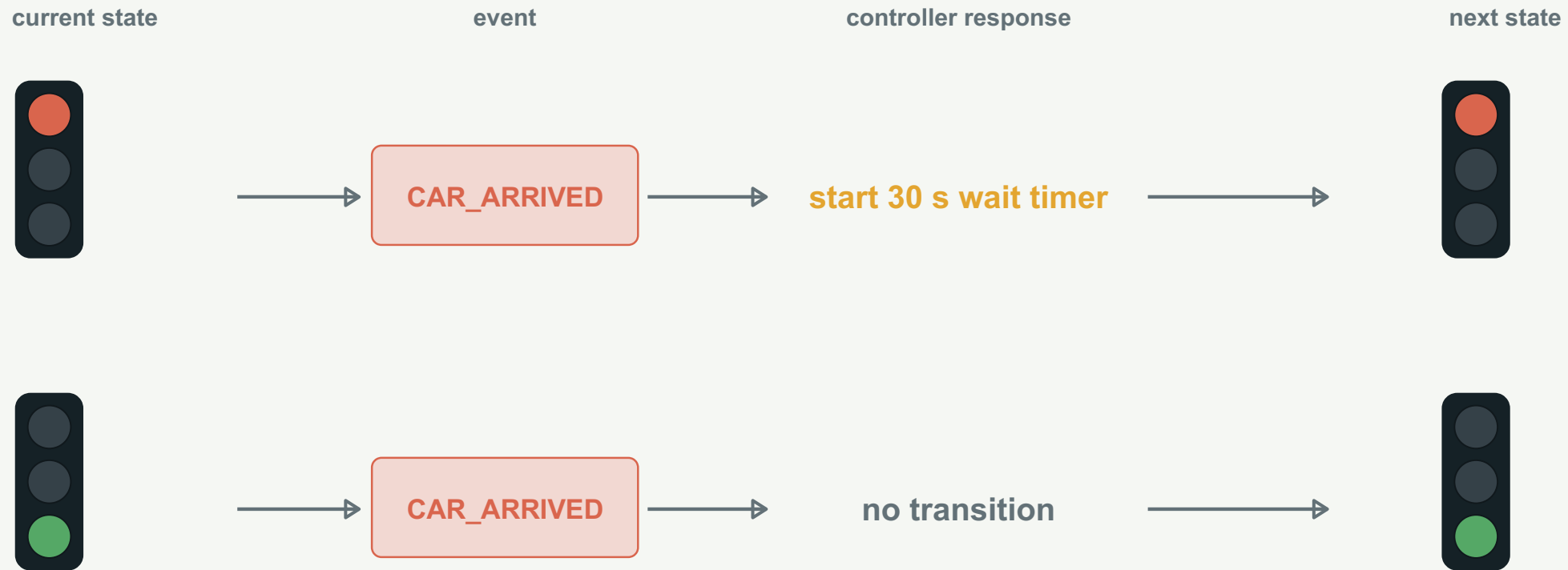
completed duration



Tap and commit are known on release.

Cancel fires while held, once. Releasing afterward emits nothing.

Events and Current State



State Representation

Independent flags

```
bool redOn   = true;  
bool yellowOn = true;  
bool greenOn = true;
```



contradictory combination is representable

One state variable

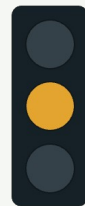
```
enum State {  
    RED,  
    GREEN,  
    YELLOW  
};  
  
State state = RED;
```



RED



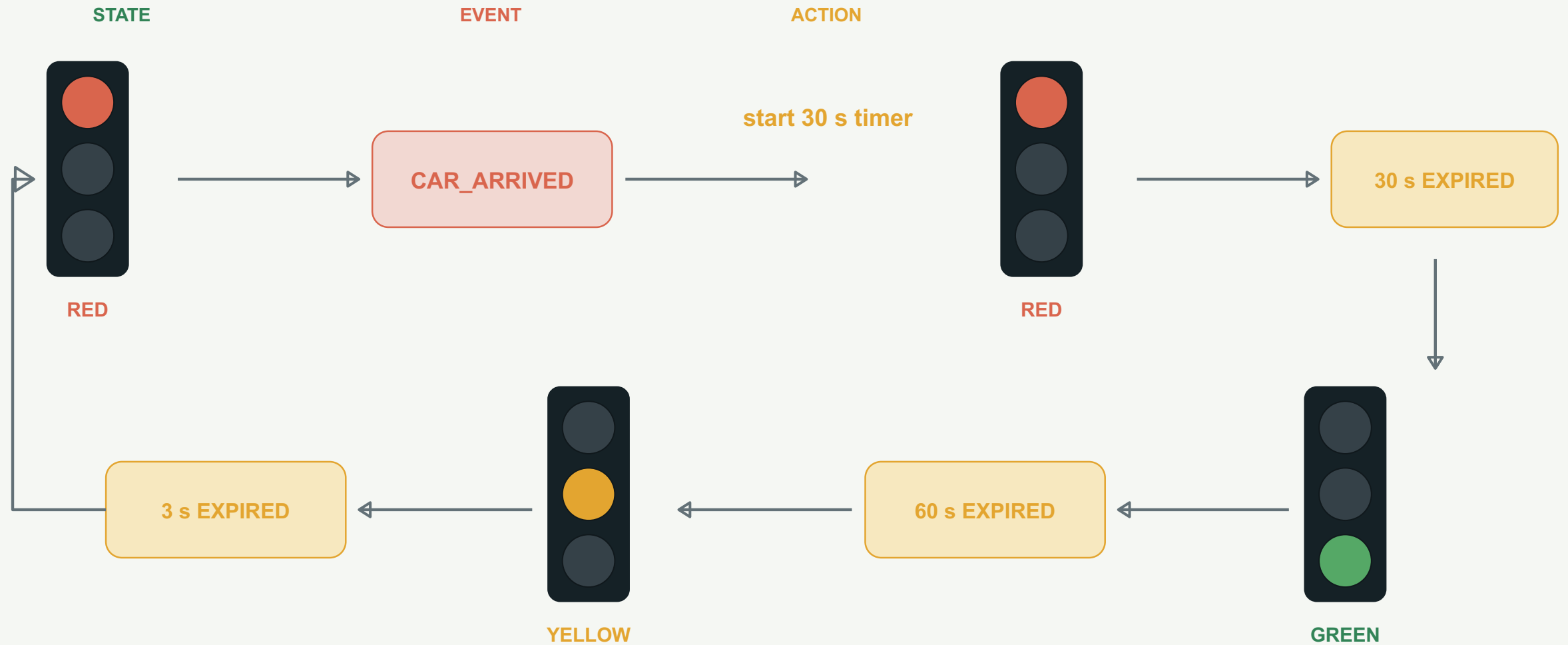
GREEN



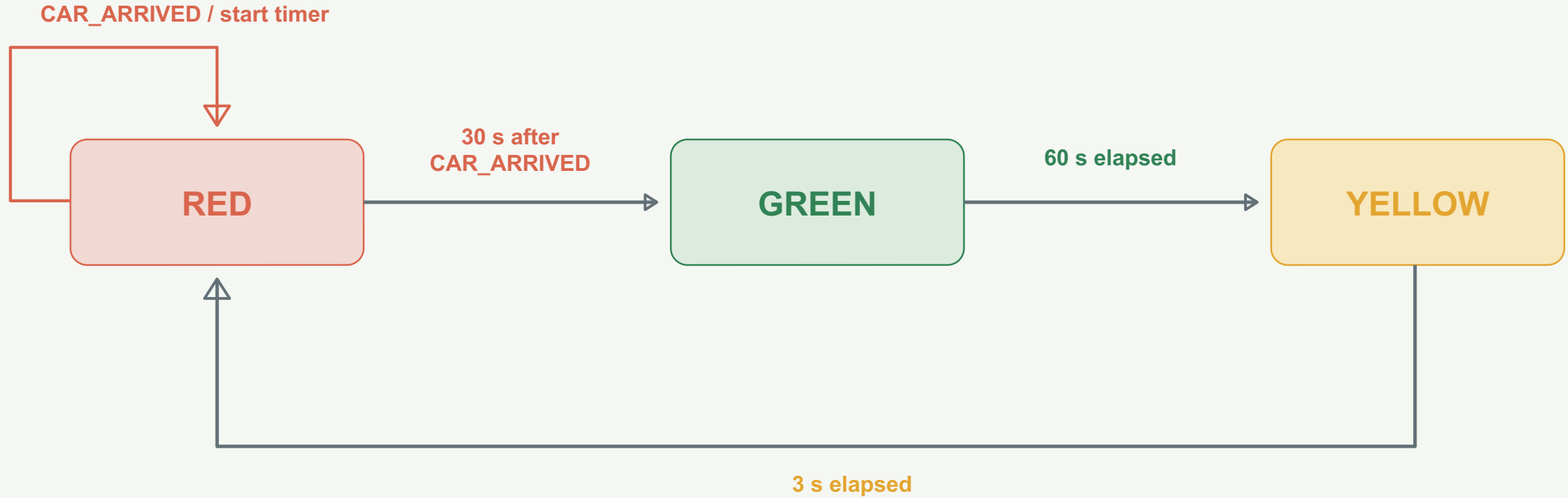
YELLOW

one variable stores one valid mode

States, Events, and Actions



State Diagram



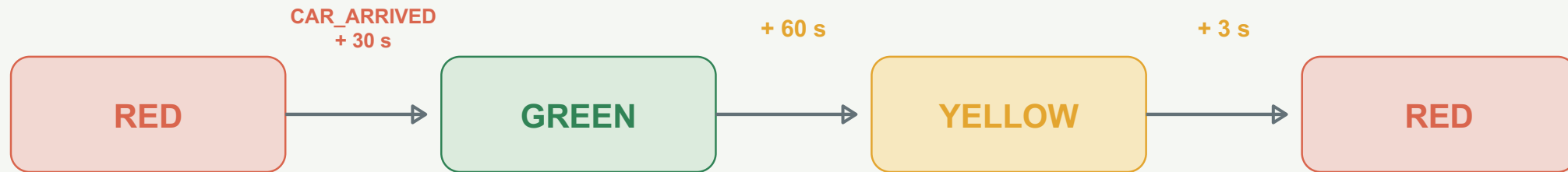
State Transition Table

CURRENT STATE	EVENT	ACTION	NEXT STATE
RED	CAR_ARRIVED	start 30 s timer	RED
RED	30_S_EXPIRED	start 60 s timer	GREEN
GREEN	60_S_EXPIRED	start 3 s timer	YELLOW
YELLOW	3_S_EXPIRED	none	RED

No matching row: remain in the current state.

Tracing a State Machine

Trace only from the current state



**During GREEN, another CAR_ARRIVED event occurs.
Which transition applies?**

No matching GREEN + CAR_ARRIVED row: remain GREEN.

Implementing a State Machine

```
switch (state) {  
  case RED:  
    if (carArrived) waitStarted = now;  
    if (waitDone) enter(GREEN, now);  
    break;  
  
  case GREEN:  
    if (elapsed(now, stateStarted, 60000))  
      enter(YELLOW, now);  
    break;  
  
  case YELLOW:  
    if (elapsed(now, stateStarted, 3000))  
      enter(RED, now);  
    break;  
}
```

Entry actions

Entry function is used to transition to a new state.

1 Transition condition met

waitDone == true



2 Enter new state

enter(GREEN, now);



3 Initialize state

**state = GREEN;
stateStarted = now;**

```
enum State { RED, GREEN, YELLOW };  
  
State state = RED;  
uint32_t stateStarted = 0;  
  
void enter(State next, uint32_t now) {  
    state = next;  
    stateStarted = now;  
}
```

Implementing a State Machine

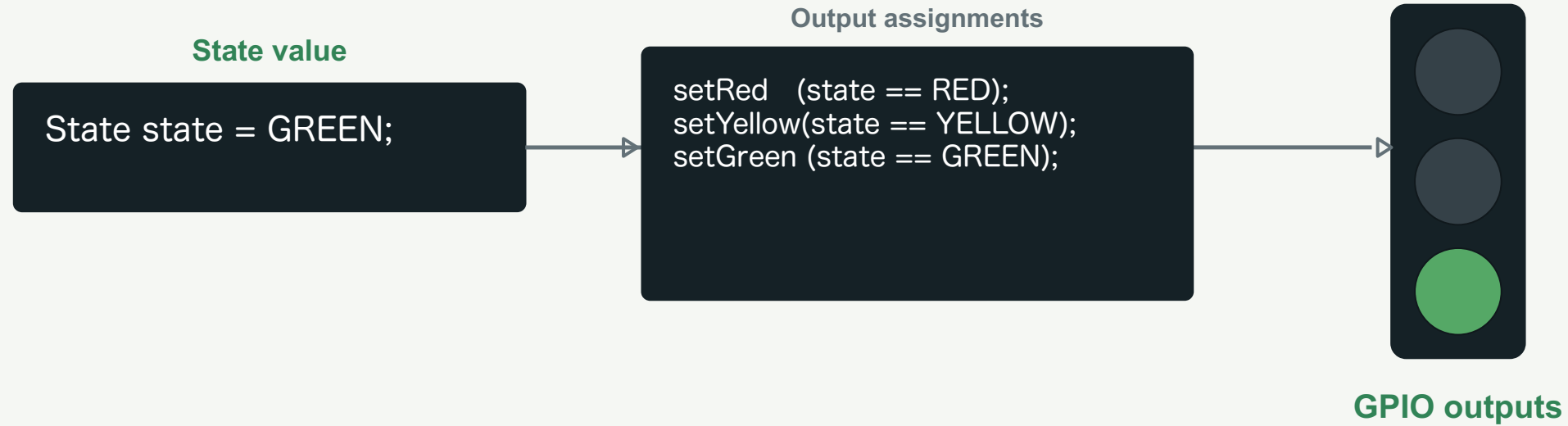
```
enum State { RED, GREEN, YELLOW };
```

```
State state = RED;  
uint32_t stateStarted = 0;
```

```
void enter(State next, uint32_t now) {  
    state = next;  
    stateStarted = now;  
}
```

```
switch (state) {  
    case RED:  
        if (carArrived) waitStarted = now;  
        if (waitDone) enter(GREEN, now);  
        break;  
  
    case GREEN:  
        if (elapsed(now, stateStarted, 60000))  
            enter(YELLOW, now);  
        break;  
  
    case YELLOW:  
        if (elapsed(now, stateStarted, 3000))  
            enter(RED, now);  
        break;  
}
```

Outputs from state



State transitions and outputs

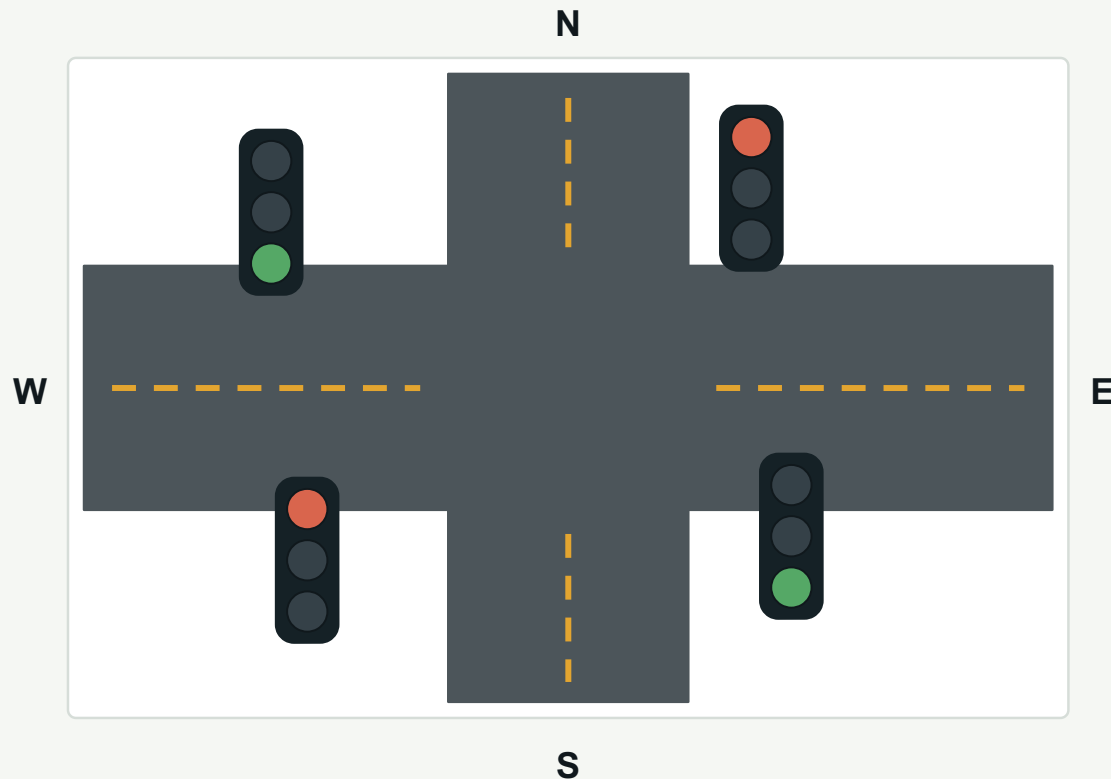
```
void loop() {
  uint32_t now = millis();
  readEvents(now); // update qualified events

  // Transition logic
  switch (state) {
    case RED:
      if (carArrived) waitStarted = now;
      if (waitDone) enter(GREEN, now);
      break;
    case GREEN:
      if (elapsed(now, stateStarted, 60000))
        enter(YELLOW, now);
      break;
    case YELLOW:
      if (elapsed(now, stateStarted, 3000))
        enter(RED, now);
      break;
  }

  // Output logic uses the current state
  setRed(state == RED);
  setYellow(state == YELLOW);
  setGreen(state == GREEN);
}
```

Design Exercise: Four-Way Intersection

E/W signals are synchronized. N/S signals are synchronized.

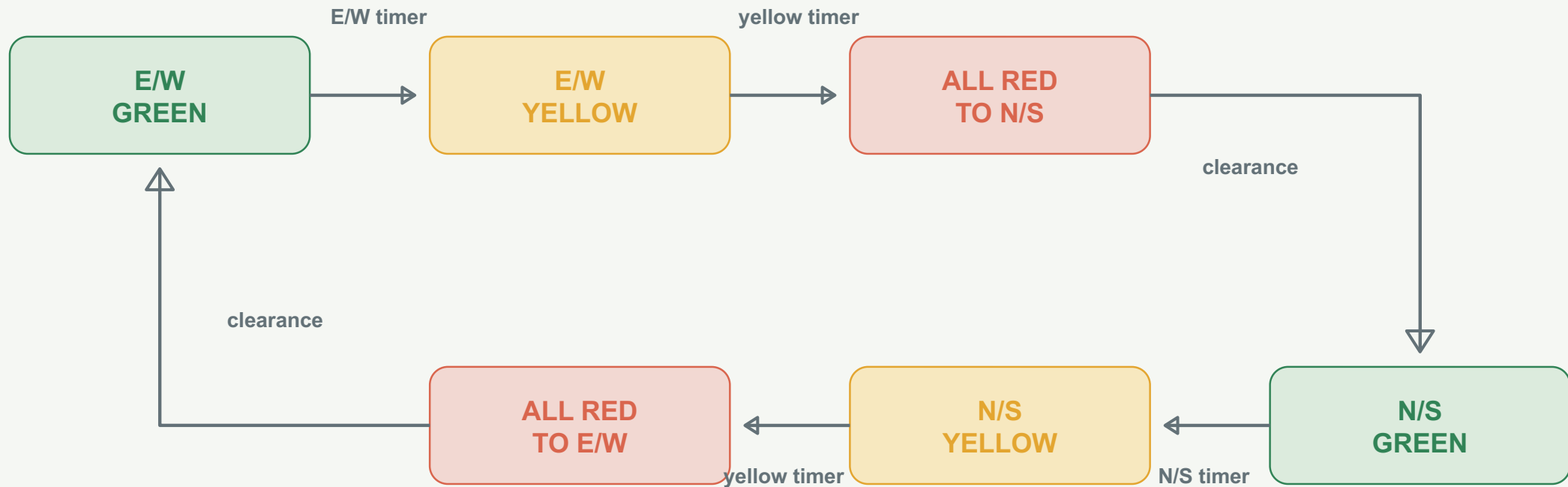


Constraints

- 1 E/W and N/S can never be green together.
- 2 Each direction passes through yellow before red.
- 3 Use an all-red clearance interval before the other direction turns green.

Name the states. Then draw the legal transitions between them.

Four-Way Intersection State Diagram



The two all-red states have identical outputs but remember different next directions.